TDP Displacement from CCTV Footage

User manual -- ground-plane homography tracking, Monte Carlo uncertainty, and two-view BLUE fusion

Covers: **01_tdp_displacement.ipynb** and **02_fusion_BLUE.ipynb**

Companion to the repository README.md and requirements.txt.

This manual documents a two-notebook pipeline that recovers the ground displacement of a Traffic Delineator Post (TDP) filmed by oblique-view CCTV cameras. Diagrams in this manual are schematic illustrations of the method, not screenshots of a specific dataset.

Contents

1. Overview and pipeline
2. Setup and requirements
3. Input data layout
4. Core concept: per-frame homography
5. Notebook 1 -- 01_tdp_displacement.ipynb
   5.1 Parameters
   5.2 Lens distortion correction (optional)
   5.3 Frame explorer (optional)
   5.4 Ground control points (GCP)
   5.5 Real-world GCP coordinates
   5.6 Sanity checks and homography verification
   5.7 Per-frame digitization of the TDP
   5.8 Review and correction
   5.9 Trajectory and Monte Carlo uncertainty
6. Notebook 2 -- 02_fusion_BLUE.ipynb
7. Keybinding cheat sheet
8. Output file reference
9. Troubleshooting and known limitations
10. Changes from the original working notebooks

1. Overview and pipeline

The pipeline estimates the ground displacement of a Traffic Delineator Post (TDP) from oblique CCTV video, using a homography computed independently on every frame (so the result is immune to camera shake), a scale anchored against surveyed ground control points, and Monte Carlo propagation of digitization and reference uncertainty. When the same event is filmed by two cameras, a second notebook synchronizes and statistically fuses the two independent traces.

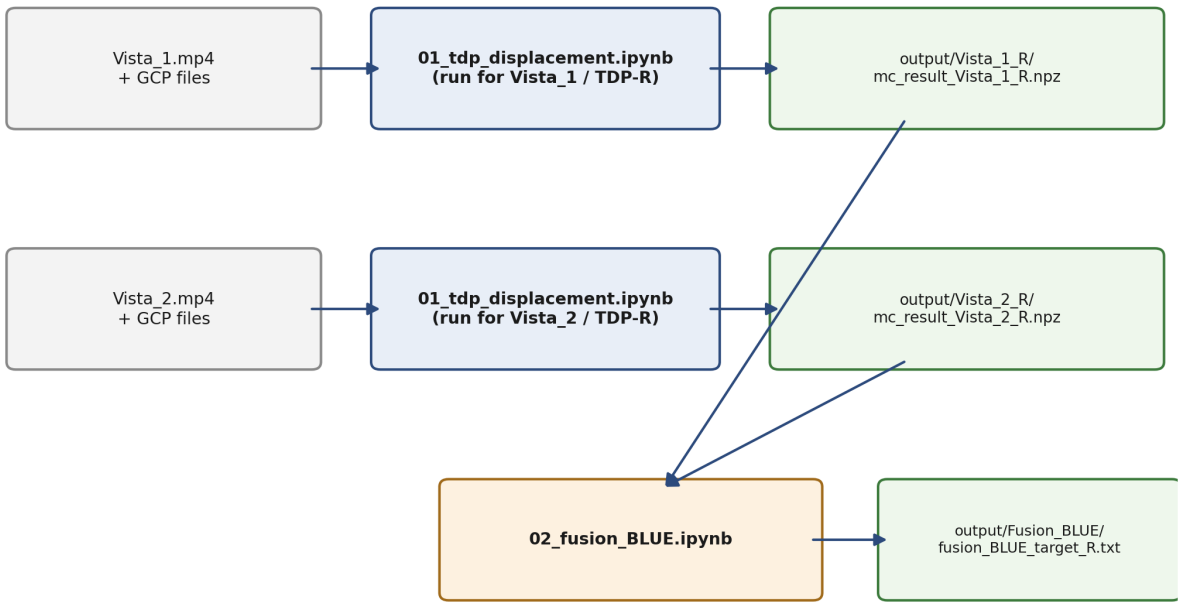


Figure 1. End-to-end pipeline: each camera view is processed independently by 01_tdp_displacement.ipynb, and the two resulting mc_result_*.npz files are combined by 02_fusion_BLUE.ipynb.

Run **01_tdp_displacement.ipynb** once per (view, target) pair -- e.g. once for Vista_1/R and once for Vista_2/R. In this project TARGET is always "R" (the right-hand TDP) and N_TARGETS is always 1, so a full run of the pipeline for one event means exactly two runs of notebook 1 followed by one run of notebook 2.

2. Setup and requirements

Python environment:

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
jupyter lab
```

Required packages: `numpy, pandas, scipy, matplotlib, opencv-python, geopandas, ipympl, ipywidgets, jupyterlab` (see requirements.txt).

Note. *ipympl provides the interactive figure backend the digitizing tools rely on. If a figure renders as a static image instead of a clickable one, install ipympl and restart the kernel. On Windows, do not switch to* `%matplotlib qt` *-- it conflicts with OpenCV's DLLs and can crash the kernel.*

3. Input data layout

Set BASE_PATH at the top of 01_tdp_displacement.ipynb to a folder with the following contents. Every output the pipeline produces is written back under this same folder, so a single edit is enough to point the whole pipeline at a new dataset.

{BASE_PATH}/

- `Vista_1.mp4, Vista_2.mp4, ...`
  - *one CCTV video per camera view*
- `ref_points_v4.gpkg`
  - *GCP in lat/long (read with geopandas)*
- `ref_points_dxf.txt`
  - *GCP in local (x, y) survey coordinates (m)*
- **`output/`** **(generated)**
  - *{View}_{Target}/mc_result_*.npz, figures/, ...*
- **`{View}_picks/`** **(generated)**

Figure 2. Expected folder layout under BASE_PATH. output/ and {View}_picks/ are created automatically by the notebooks.

4. Core concept: per-frame homography

Every sampled frame gets its own homography H, fit from that frame's clicked ground control points (GCP) and their known real-world coordinates. Because H is recomputed per frame, camera vibration or drift does not bias the trajectory -- only the accuracy of that frame's own GCP clicks matters. At least 4 non-collinear GCP are required; more GCP make the reprojection RMSE check in section 5.6 meaningful rather than an exact fit by construction.

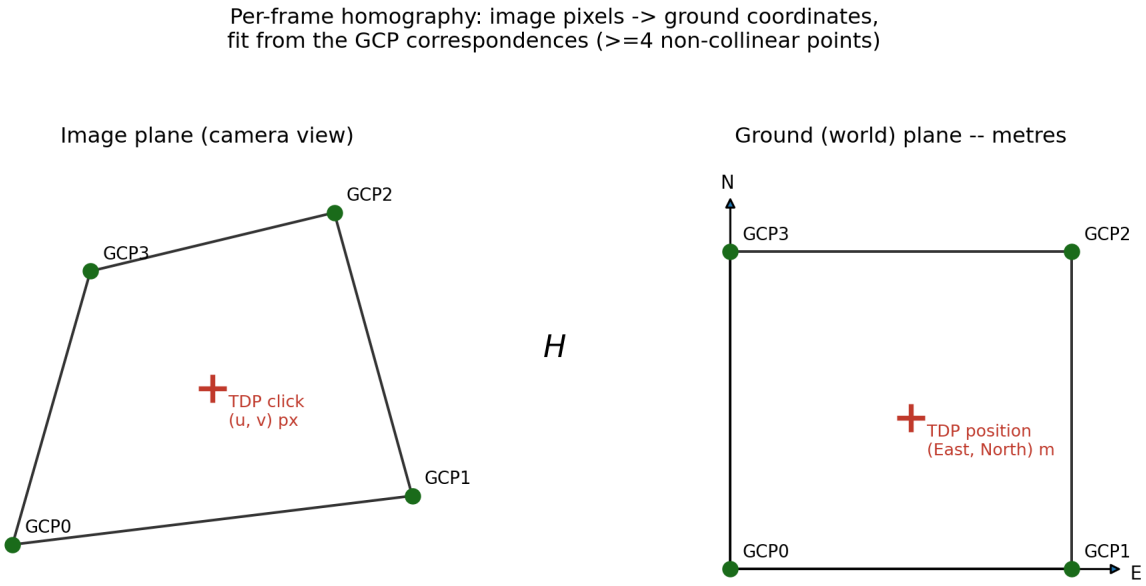


Figure 3. A frame's GCP correspondences (image pixels <-> real-world East/North) fit a homography H, which then maps the clicked TDP pixel to a ground position.

5. Notebook 1 -- 01_tdp_displacement.ipynb

Run the notebook's cells in order. Section 1 (parameters and functions) is required. Section 2 (frame explorer) and section 1b (distortion correction) are optional. Sections 7 and 8 (digitization and review) are interactive; the rest run automatically. Everything digitized is saved immediately and is resumable if the kernel restarts.

5.1 Parameters

Edit only the parameters cell for a new case; every other cell reads from it.

Parameter	Example	Meaning
BASE_PATH	**"/data/EQ"**	Base working folder (see section 3).
VIEW_NAME	**"Vista_2"**	Camera view name; used to name every output file.
TARGET	**"R"**	Target name; fixed at "R" (right TDP) in this project.
N_TARGETS	**1**	Targets tracked per frame; fixed at 1 in this project.
FRAME_REF	**76**	Reference frame used for GCP digitization and scale anchoring.
FRAME_START/END	**536 / 736**	Analysis window (ideally: just before the event, to rest).

Parameter	Example	Meaning
`N_GCP`	`7`	Number of ground control points to digitize (minimum 3, ideally 4+).
`T_STEP_S`	`0.1`	Temporal sampling step, in seconds.
`SIGMA_CLICK_PX`	`1.0`	Assumed digitization (clicking) error, in pixels.
`SIGMA_GCP_WORLD_M`	`0.01`	Residual GCP error after scale anchoring, in metres.
`N_SIM`	`300`	Number of Monte Carlo simulations.

5.2 Lens distortion correction (optional, recommended)

Wide-angle CCTV lenses curve real-world-straight lines; a planar homography cannot model that curvature, which biases results toward the frame edges. This section fits a standard OpenCV radial-distortion model (k1, k2) from several real-world-straight lines you digitize once with the StraightLinePicker tool, using cv2.undistortPoints (an exact, iterative inversion) rather than an approximate closed-form formula, which was tested and found to give 10-30% error at the edges for distortion of this magnitude.

- Check first whether it is even needed: overlay a guide line on a straight real-world edge that crosses the frame -- negligible bowing means you can skip this section.
- Mark 3-6 straight lines, 4-8 points each, spread across the frame and reaching the edges.
- The fit is cached to `<video>_distortion.json` and reused automatically on later runs -- set `FORCE_RECALIBRATE = True` to redo it.
- Recommended: digitize one extra straight line NOT used in the fit and confirm it also comes out straight, to check the model is not overfitting to the calibration lines.

5.3 Frame explorer (optional)

A lightweight, matplotlib-free slider (JPEG widget) to scrub through the video and note frame indices for FRAME_START / FRAME_END / FRAME_REF.

5.4 Ground control points (GCP)

Click N_GCP coplanar, well-separated, non-collinear points on the reference frame (e.g. post bases, tile corners, fence corners). Remember the click order -- you map it to real coordinates in the next section.

5.5 Real-world GCP coordinates

GCP from a satellite-derived `.gpkg` file are projected to local UTM metres; a second, survey-grade set of points from a `.txt` (extracted from a DXF) is rotated and translated onto the same frame using a shared baseline azimuth, then used as the actual GCP source (better absolute accuracy than the satellite points). `REF_POINT_ORDER` maps each GCP click (section 5.4, in your own click order) to the matching point index in this file.

5.6 Sanity checks and homography verification

Two independent checks before digitizing the full sequence: (1) click pairs of points on ground features of known size (kerb width, tile footprint, ...) and confirm the measured distance is sensible; (2) an automatic check -- GCP reprojection RMSE and all pairwise GCP distances, real vs. homography-projected. A discrepancy above 10% on

either check usually means a wrong REF_POINT_ORDER or non-coplanar GCP -- fix it before continuing.

5.7 Per-frame digitization of the TDP

For every sampled frame: the N_GCP GCP in the same order as section 5.4, then the TDP itself. The TDP uses a reusable **template** (an outline plus one anchor point) so that each frame only needs the template dragged into place and accepted, rather than the whole outline re-clicked.

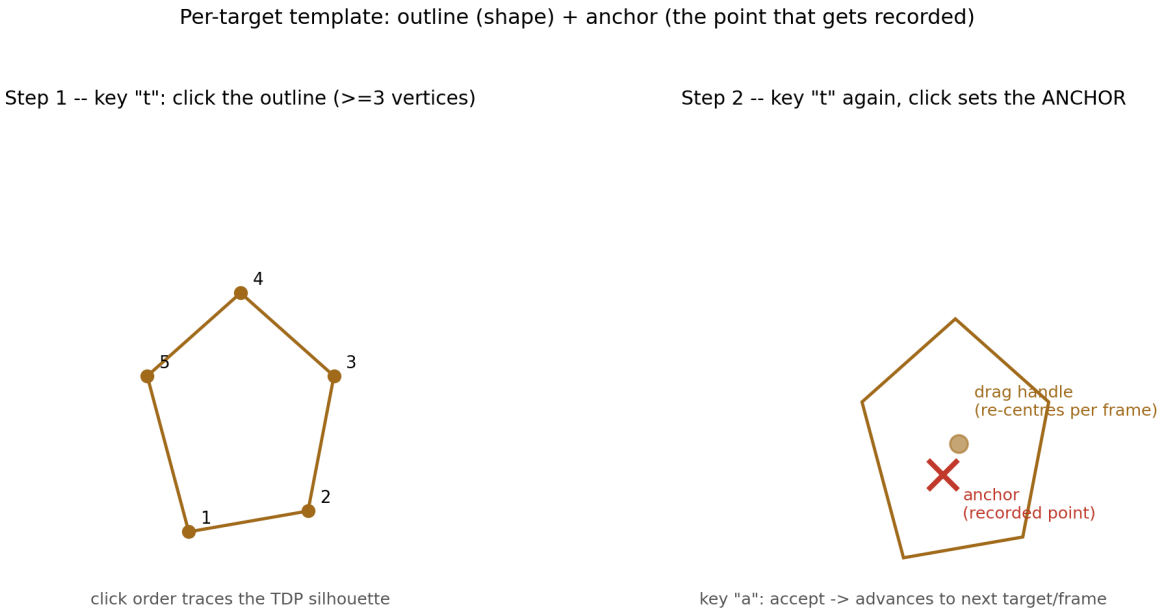


Figure 4. Defining a target's template: trace its outline, then click the anchor -- the single point that actually gets recorded as the target's position on every frame.

The template can be predefined with the standalone TemplateEditor tool before section 5.7, or defined inline during digitization with key t. All digitization progress is written immediately to `clicks.json` and an annotated JPEG per frame, under the `{View}_picks/` folder -- safe to stop and resume.

5.8 Review and correction (optional)

Step through already-digitized frames and drag any point (GCP or TDP) to correct it; the per-frame GCP reprojection RMSE recomputes live in the title. This is also where you fix any frames the pipeline auto-migrated from an older single-target click format.

5.9 Trajectory and Monte Carlo uncertainty

Section 9 converts every digitized frame to world coordinates with that frame's own homography, reports the per-frame GCP RMSE as a quality flag, and saves the raw trace to CSV and to a plain text file matching what notebook 2 expects.

Section 10 is this notebook's **final product**: it perturbs every click (GCP and TDP) and the GCP world coordinates N_SIM times, applies a zero-phase Butterworth low-pass filter, and derives a per-frame position covariance from the resulting ensemble. It saves

```
output/{View}_{Target}/mc_result_{View}_{Target}.npz
```

containing the filtered trace, the raw (unfiltered) trace, and the per-frame 2x2 covariance -- the only file notebook 2 needs.

6. Notebook 2 -- 02_fusion_BLUE.ipynb

Combines the mc_result_*.npz output of two views of the same TDP into a single, more certain trace, in two steps.

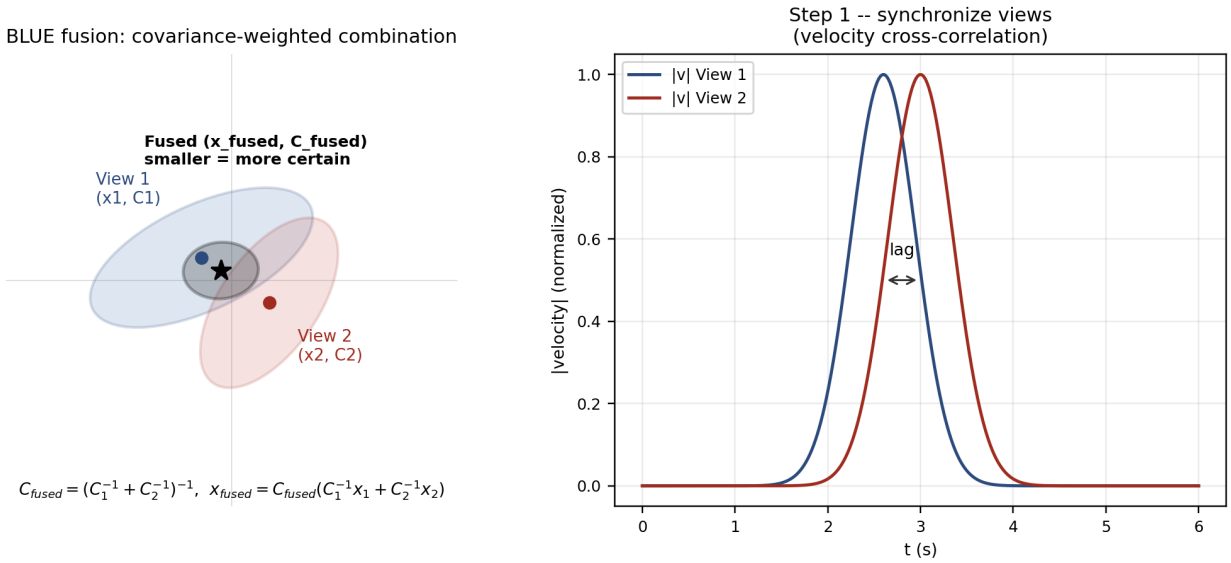


Figure 5. Left: BLUE combines two noisy estimates weighted by the inverse of their covariance -- the fused ellipse is never larger than either input. Right: the two views are synchronized first, by cross-correlating their TDP velocity magnitude.

- **Synchronization** -- the time lag between View_2 and View_1 is found by cross-correlating velocity magnitude (sharper peak than displacement) and summing the correlation curves across all targets before finding the joint maximum, since both targets share one true lag. A 3-point quadratic fit around the peak gives sub-frame precision; the correlation peak's 90%-width is reported as the lag's real resolution.
- **Per-frame BLUE fusion** -- once synced, each view is resampled onto a common time grid and combined frame-by-frame as C_fused = (C1^-1 + C2^-1)^-1, x_fused = C_fused (C1^-1 x1 + C2^-1 x2), using the full 2x2 covariance rather than just the diagonal variances.

Final outputs, one file per target:

```
output/Fusion_BLUE/fusion_BLUE_target_{Target}.txt
output/Fusion_BLUE/figures/*.png   (+ figures_manifest.json)
```

7. Keybinding cheat sheet

All interactive tools share the same conventions where possible.

PointPicker (GCP on the reference frame)

Key	Action
`click`	Register the next point (up to N).
`z`	Undo the last clicked point.
`l`	Add a draggable guide line (visual aid; not recorded as data).
`d`	Remove the last guide line.

StraightLinePicker (distortion calibration)

Key	Action
`click`	Add a point to the current line.
`n`	Finish the current line, start a new one.
`z`	Undo the last point (or the last finished line).

SequencePicker (per-frame GCP + TDP digitization)

Key	Action
`click`	Register the next point (GCP, then target).
`n`	Skip the whole current frame.
`z`	Undo the last click, or go back to the previous frame.
`t`	Toggle template definition for the current target: outline, then anchor.
`a`	Accept the active template's anchor as this frame's target position.
`arrows`	Nudge the current target's template by 1 px (shift = 5 px).
`l / d`	Add / remove a guide line.

FrameReviewer (section 5.8)

Key	Action
`. / ,`	Next / previous frame.
`drag point`	Correct any GCP or target position; saved on release.
`a (/ b)`	Snap target 1 (/ target 2, if N_TARGETS > 1) to its template anchor.
`arrows`	Nudge the active template by 1 px (shift = 5 px).
`l / d`	Add / remove a guide line.

SegmentMeasurer (section 5.6 sanity check)

Key	Action
`click, click`	Every 2 clicks define one measured ground segment.
`z`	Undo the pending point, or the last completed segment.

TemplateEditor (standalone template definition)

Key	Action
`click`	Add an outline vertex (outline mode) or set the anchor (anchor mode).
`t`	Close the outline (needs >= 3 vertices) and switch to anchor mode.
`z`	Undo the last vertex, or go back from anchor to outline mode.
`e`	Load the existing template's vertices for editing.
`arrows`	Nudge the fixed anchor by 1 px (shift = 5 px).

8. Output file reference

Parameter	Example	Meaning
`<video>_distortion.json`	`next to video`	Cached lens-distortion fit (k1, k2).
`{View}_picks/{Target}/`	`under BASE_PATH`	clicks.json, templates.json, annotated JPEGs.
`<video>_tdp_trajectory.csv`	`next to video`	Raw per-frame trajectory (frame, t, pixel and world coords, RMSE).
`{View}_{Target}_raw_trace.txt`	`output/{View}_{Target}/`	Raw (t, East, North) trace; read by fusion_BLUE.
`mc_result_{View}_{Target}.npz`	`output/{View}_{Target}/`	Filtered trace + per-frame covariance; the key handoff file.
`figures/*.png` + `figures_manifest.json`	`output/{View}_{Target}/`	Every figure the notebook saves, with captions.
`fusion_BLUE_target_{Target}.txt`	`output/Fusion_BLUE/`	Final fused (t, East, North, sigma_E, sigma_N).

9. Troubleshooting and known limitations

- **Static, non-interactive figures** -- install/upgrade ipympl and restart the kernel; confirm the first cell prints an interactive backend, not "inline" or "agg".
- **Kernel crashes on a figure** -- do not switch to %matplotlib qt on Windows; it conflicts with OpenCV's DLLs.
- **GCP reprojection RMSE looks too good with 4 GCP** -- with exactly 4 non-collinear points the homography fit is exact by construction; use 5 or more for the RMSE check to be informative.
- **A ground sanity-check measurement looks wrong** (e.g. an implausible sidewalk width) -- check REF_POINT_ORDER first, then GCP coplanarity, then the scale anchoring.
- **Resuming after a kernel restart** -- re-run cells top to bottom; digitization and distortion calibration reload their cached progress automatically and skip the interactive step if nothing is missing.
- This pipeline only tracks one TDP (N_TARGETS = 1). The underlying tools do support multiple simultaneous targets (a second colour, key b, etc.) but that path is untested in this project and left in place only because removing it would be a much larger change than trimming unused sections.

10. Changes from the original working notebooks

This cleaned version differs from the original working copies in a few ways worth knowing about if you compare against an older draft:

- Sections that computed velocity, directional PGD/PGV peaks, and an annotated trajectory video were removed from notebook 1 -- not needed for the displacement trace that fusion_BLUE consumes. They are straightforward to rebuild from the Monte Carlo position ensemble already produced in section 5.9 if needed again.
- **Bug fix:** N_TARGETS is now set in the parameters cell; previously it was defined one cell after the template-editor cell that already depended on it, which raised a NameError on a clean run.
- **Bug fix:** a duplicated pair of HANDLE_TOL_PX / arrow-key-nudge constant definitions was removed.
- Hardcoded, machine-specific Windows paths were replaced by a single BASE_PATH variable used consistently via cross-platform path joins.
- Output filenames were made consistent between the two notebooks (e.g. `{View}_{Target}_raw_trace.txt`) so fusion_BLUE reads exactly what notebook 1 writes.
- TARGETS in fusion_BLUE is fixed to ["R"] and TARGET/N_TARGETS in notebook 1 are fixed to "R" / 1, reflecting that this project only ever tracks the right-hand TDP, in both camera views.

Note. *insert_figures_into_report.py, referenced in a couple of docstrings as the script that consumes figures_manifest.json to populate a Word report, is part of a separate reporting step and is not included in this repository.*